

Running CUDA-Only Applications on an AMD GPU

Why AMD users can't just install Octane — and the tools that are trying to change that.

A teacher's companion to the Ollama + ROCm on a System76 Pangolin15 runbook. Read that one first. This guide picks up where it left off.

TL;DR

If you followed the Pangolin15 ROCm guide and now want to run CUDA-only creative apps (think OctaneRender, Blender Cycles via CUDA, or PhysX games), this document explains:

- Why CUDA apps don't "just work" after you install ROCm
- Three tools that bridge the gap — ZLUDA, SCALE, and HIP/hipify
- Which tool fits which situation (spoiler: there is no magic bullet)
- What Octane specifically needs, and whether it is currently achievable

Current status on the AMD Radeon 780M (gfx1103):

App	Tool	Status	Notes
Blender Cycles (CUDA mode)	ZLUDA 6	✔ Works	Textures now supported as of v6 ^{[1][2]}
PhysX (legacy games)	ZLUDA 6 pre-alpha	⚠ Experimental	32-bit only, fluid sims can be glitchy ^[^2]
PyTorch / ML workloads	ZLUDA 6	✔ Improving	Steady PRs improving instruction coverage ^[^2]
OctaneRender (Windows/Linux)	None confirmed	✖ Not yet	CUDA-only; OTOY still working on AMD support ^{[3][4]}
Custom CUDA source code	SCALE	✔ If you have source	gfx1102 supported; gfx1103 ad-hoc ^[^5]
Porting CUDA code to AMD	HIP/hipify	✔ Developer tool	Requires code access and recompile ^{[6][7]}

The History-Teacher Backstory

Class, let us talk about why the GPU computing world is one giant proprietary mess — and how a handful of open-source rebels are trying to fix it.

The CUDA monopoly. In 2006, NVIDIA introduced CUDA (Compute Unified Device Architecture), a programming platform that lets developers write software that runs directly on a GPU. It was brilliant, it was fast, and it had a catch: it only runs on NVIDIA hardware. Developers loved it so much that entire industries — 3D rendering, scientific computing, AI/ML — built their software on top of it. OctaneRender, for example, was built on CUDA from the ground up.^{[8][9]}

AMD's response: ROCm. AMD built its own compute stack called ROCm (Radeon Open Compute), with a CUDA-like language called HIP (Heterogeneous-computing Interface for Portability). It is good software, but ROCm is not CUDA. Apps written for CUDA do not run on ROCm. They are two different languages spoken by two different countries, with no automatic translator.[⁸]

The runbook you just read. The Pangolin15 guide used `HSA_OVERRIDE_GFX_VERSION=11.0.2` to trick ROCm into thinking your gfx1103 chip is a supported gfx1102. That solved the "ROCm won't talk to my iGPU" problem. It did *not* solve the "this app speaks CUDA and ROCm is speaking something else" problem. Those are two separate dragons.[¹⁰][11]

So what are the translation tools? Three projects are tackling the CUDA-to-AMD translation problem, each in a very different way. Think of them as three different teaching strategies for getting a Spanish-speaking student to pass an English exam.

The Three Tools

Tool 1: ZLUDA — The Runtime Impersonator

What it is: ZLUDA is an open-source project by developer Andrzej Janik. Instead of rewriting anything, it intercepts CUDA calls at runtime and secretly routes them to AMD's ROCm underneath. The app thinks it is talking to an NVIDIA GPU. ZLUDA plays the role of NVIDIA and quietly passes the work to AMD.[¹¹][12]

How it works in one sentence: You launch an existing CUDA binary through ZLUDA, and it substitutes itself as the CUDA runtime — no code changes, no recompile.[¹³]

Current status (as of June 2026): ZLUDA 6 was released in late June 2026. It now supports 32-bit PhysX, Blender textures (meaning Blender Cycles renders correctly), and has significantly improved Windows support. However, commercial funding was just lost again, so the project is back to being a weekend hobby for Janik. Development continues but at reduced pace.[¹⁴][2]

What it can and cannot do:

- ✓ Blender Cycles — fully working with textures[²]
- ✓ Several ML frameworks (PyTorch, others)[²]
- ✓ 32-bit PhysX (pre-alpha, some games)[²]
- ☐ Arnold renderer — partial[¹³]
- ✗ OctaneRender — not confirmed working, and OTOY has not endorsed it[³]
- ✗ Fluid simulation physics — still glitchy[²]

Legal note for the classroom: NVIDIA's license terms for the CUDA Toolkit forbid using it to develop technologies that translate CUDA output to "target a non-NVIDIA platform." This created controversy around ZLUDA v3 and has been a recurring issue for the project. ZLUDA gets around this by being a clean-room implementation, but the legal fog never fully cleared.[¹³]

Install path (Linux): Requires ROCm already installed (so yes, the Pangolin15 runbook is step 0). Download the ZLUDA binary from the project's GitHub releases page, then wrap your CUDA application like:

```
# General pattern – wrap your target app binary with ZLUDA
LD_LIBRARY_PATH=/path/to/zluda:$LD_LIBRARY_PATH ./your-cuda-app
```

The exact launch method varies by application. Blender has a specific integration path detailed in the ZLUDA repository's wiki.

Tool 2: SCALE — The Compiler Impostor

What it is: SCALE, built by London-based startup Spectral Compute, takes a different approach. Instead of intercepting runtime calls, it *recompiles* your CUDA source code natively for AMD hardware. It acts as a drop-in replacement for `nvcc` (NVIDIA's CUDA compiler).^{[15][16]}

How it works in one sentence: Give SCALE your CUDA source code, and it produces an AMD binary — no translation layer needed at runtime, no NVIDIA code involved.^{[17][5]}

Why this is a bigger deal than ZLUDA (for developers): SCALE is a "clean room" implementation of CUDA — it does not touch NVIDIA's code at all, avoiding the legal headaches. It also handles inline PTX assembly (`asm`) which many translators cannot.^{[15][17]}

What GPU targets does it officially support?

Target	Architecture	Status
gfx900	Vega 10 (GCN 5.0)	✔ Nightly tested ^[^5]
gfx1030	Navi 21 (RDNA 2.0)	✔ Nightly tested ^[^5]
gfx1100	Navi 31 (RDNA 3.0)	✔ Nightly tested ^[^5]
gfx1102	RDNA 3 (close cousin)	☒ Ad-hoc tested ^[^5]
gfx1103	Radeon 780M (your iGPU)	☒ Untested — may work with override ^[^5]

Since `gfx1103` is not an official SCALE target, you would likely need to combine it with the same `HSA_OVERRIDE_GFX_VERSION=11.0.2` trick from the Pangolin15 runbook — telling SCALE to compile for `gfx1102` and trusting that the binaries run on `gfx1103`.

The big catch: SCALE requires access to the application's *source code*. If you want to run a pre-compiled, closed-source binary like OctaneRender, SCALE cannot help you — there is no source to recompile. SCALE is a developer tool, not a user tool.^{[5][15]}

Install path (Linux):

```
# Download and extract SCALE (free tier available)
wget https://pkgs.scale-lang.com/tar/scale-free-1.3.1-amd64.tar.xz
tar xf scale-free-1.3.1-amd64.tar.xz
```

```
# Activate the virtual environment for your target GPU ISA
# For the 780M, use gfx1102 and rely on your HSA override
source scale-free-1.3.1-Linux/bin/scaleenv gfx1102

# Now use 'scalecc' wherever you would have used 'nvcc'
scalecc -o my_app my_cuda_source.cu
```

SCALE is available under a free software license, but it is *not* open-source.^{[18][15]}

Tool 3: HIP / hipify — The Code Porter

What it is: HIP (Heterogeneous-computing Interface for Portability) is AMD's own CUDA-like programming language. `hipify` is a set of tools that *automatically converts CUDA source code into HIP source code*, so developers can recompile their app natively for AMD hardware.^{[19][7][^20]}

How it works in one sentence: Run `hipify-perl` or `hipify-clang` on your CUDA source files and they come out as HIP files, ready to compile with AMD's `hipcc` compiler.^{[6][20]}

This is a developer migration tool, not a user shortcut. HIP is appropriate when:

- You are porting your *own* CUDA project to AMD
- You want maximum native performance on AMD hardware
- You are prepared to fix the ~5-10% of CUDA constructs `hipify` does not convert automatically^[^19]

Install: `hipify` is bundled with ROCm — if you followed the Pangolin15 runbook, you already have it.

```
# Convert a CUDA source file to HIP
hipify-perl my_cuda_file.cu > my_hip_file.hip.cpp

# Compile the HIP file for AMD
hipcc -o my_app my_hip_file.hip.cpp
```

For a full project, use `hipify-perl` or `hipify-clang` with CMake integration. AMD's official HIP porting guide covers the details.^{[7][20]}

The OctaneRender Question

The original YouTube comment asked specifically about Octane. Here is the honest answer.

Why Octane is hard: OctaneRender on Windows and Linux requires a CUDA-enabled NVIDIA GPU to render. It is a closed-source, pre-compiled binary. This immediately rules out SCALE (no source code to compile) and HIP/hipify (same reason). ZLUDA is the only option — and there is no confirmed report of OctaneRender running on AMD via ZLUDA as of July 2026.^{[21][3]}

What OTOY (Octane's maker) has said: OTOY's FAQ says "work is ongoing to assimilate GPUs from AMD and Intel for future versions." Historically, OTOY built an AMD version but needed

full driver support from AMD that never arrived. Octane X, the macOS version, fully supports AMD GPUs using Apple's Metal API — but Metal is macOS-only.^{[4][22][3][21]}

The closest working alternative today: Blender Cycles, which is free and open-source, now works on AMD via ZLUDA 6 — including textures. For GPU-accelerated rendering workflows on AMD hardware, Blender + Cycles is the most reliable path until Octane adds native AMD support.^{[23][2]}

How It All Connects to the Pangolin15 Runbook

Think of the Pangolin15 guide as **Level 1**: getting AMD's own compute brain (ROCm) to recognize and talk to the 780M iGPU. That was already a hack — a friendly one, but a hack.^{[11][10]}

ZLUDA, SCALE, and HIP are **Level 2**: getting software that speaks CUDA to work with that compute brain. The two levels stack on top of each other:

```
Your CUDA App
  |
  ▼
[ZLUDA] → intercepts CUDA calls → ROCm (with HSA_OVERRIDE=11.0.2) → gfx1103 hardware
OR
[SCALE] → compiles CUDA source → AMD native binary → ROCm → gfx1103 hardware
OR
[HIP]   → translates CUDA source → HIP source → hipcc → gfx1103 hardware
```

The Pangolin15 runbook already sets up the bottom half of that stack (ROCm + the gfx override). These tools plug into the top.^{[24][10][^11]}

Decision Tree: Which Tool Should You Use?

```
Do you have the application's source code?
├─ NO → Use ZLUDA (runtime wrapper, no source needed)
│     └─ App is a pre-compiled binary (Octane, etc.)?
│         └─ Check ZLUDA compatibility list on GitHub first
│             └─ If not listed: follow the project's Discord for updates
└─ YES → Do you want to stay in CUDA land or move to AMD native?
        ├── Stay in CUDA (single codebase, compile for both)
        │   └─ Use SCALE
        └─ Port to AMD natively (best long-term AMD performance)
            └─ Use hipify + HIP
```

Prerequisites (Same as the Pangolin15 Runbook)

Before any of these tools will work on your 780M, you need everything from the Pangolin15 runbook:

- ROCm userspace installed (rocm-hip-libraries, rocm-libs)[^24]
- GTT memory allocated (32GB via kernel boot parameters)[^11]
- HSA_OVERRIDE_GFX_VERSION=11.0.2 set in your environment[^10]
- User added to render and video groups[^11]
- **Do NOT install amdgpu-dkms** — it will conflict with the System76 kernel module[^11]

Verify ROCm is still working before attempting any of the tools below:

```
/opt/rocm/bin/rocminfo | grep -E "Marketing Name|Device Type|Name"
# Should show AMD Radeon 780M Graphics with Device Type: GPU
```

Troubleshooting

ZLUDA: App still falls back to CPU or crashes

```
# Check that ZLUDA is being loaded first
LD_PRELOAD=/path/to/zluda/libcuda.so.1 ./your-app --version
```

If the app checks for an NVIDIA device ID specifically (some do), ZLUDA may fail that check. There is no current workaround for apps that perform hardware fingerprinting.

SCALE: Compilation succeeds but binary crashes on 780M

SCALE does not officially test gfx1103. Activate the gfx1102 environment and ensure HSA_OVERRIDE_GFX_VERSION=11.0.2 is still set in the same shell:

```
export HSA_OVERRIDE_GFX_VERSION=11.0.2
source scale-free-1.3.1-Linux/bin/scaleenv gfx1102
./your-compiled-app
```

hipify: Output file has compilation errors

About 5-10% of CUDA constructs do not map cleanly to HIP. Common culprits:[^19]

- thrust:: device code — switch to rocThrust
- Texture objects with uncommon sampling modes
- Warp-level primitives — HIP has equivalents, but function names differ

Run hipify-clang (the more accurate but slower tool) instead of hipify-perl for stubborn files.

Health Check (run first when something breaks)

```
# 1. GTT allocation still in place?
cat /sys/module/ttm/parameters/pages_limit
# Expect: 8388608

# 2. ROCm still sees the GPU?
/opt/rocm/bin/rocm_info | grep gfx1103
# Expect: two lines with gfx1103

# 3. HSA override active?
echo $HSA_OVERRIDE_GFX_VERSION
# Expect: 11.0.2

# 4. render/video group membership?
groups | grep -E "render|video"
```

References and Further Reading

- ZLUDA GitHub — source code, release builds, compatibility notes:
<https://github.com/vosen/ZLUDA>
- ZLUDA Q1/Q2 2026 update post (developer's own words):
<https://vosen.github.io/ZLUDA/blog/zluda-update-q1q2-2026/>^[2]
- SCALE documentation: <https://docs.scale-lang.com/>^[5]
- Spectral Compute SCALE announcement (Tom's Hardware):
<https://www.tomshardware.com/tech-industry/new-scale-tool-enables-cuda-applications-to-run-on-amd-gpus>^[16]
- AMD HIP Porting Guide (official ROCm docs):
https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_porting_guide.html^[7]
- AMD GPU Lab Notes — hipify in practice: <https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-hipify-readme/>^[20]
- llm-tracker.info — community notes on gfx1103 compatibility: <https://llm-tracker.info/howto/AMD-GPUs>^[25]
- OctaneRender AMD status FAQ: <https://home.otoy.com/render/octane-render/faqs/>^[3]

Closing Thought

The Pangolin15 runbook ended with: *"The Sega Master System in your laptop is now playing NES cartridges."*

This guide is the follow-up chapter: **some NES cartridges have a hardware lockout chip inside that checks for a real NES before booting.** ZLUDA is the people who reversed-engineered that lockout chip and built a bypass. SCALE is the people who wrote their own cartridge standard from scratch. HIP is the people who asked the game developer to release an official Master System port.

OTOY has not released an official AMD port for Windows/Linux Octane yet. The lockout chip is still in the cartridge. But the community is working on it — and the groundwork you laid in the Pangolin15 guide is exactly what they need to stand on.^[4]^[3]^[^2]

Class dismissed. Do your homework.

Guide licensed CC BY-SA 4.0. Companion to the Ollama + ROCm on a System76 Pangolin15 runbook by Tim Dickey.

References

1. [ZLUDA 6: Running CUDA on AMD GPUs Is Now a Hobby ...](#) - ZLUDA 6 lets AMD GPUs run unmodified CUDA applications, adding PhysX support, Blender textures, and ...
2. [ZLUDA update Q1&Q2 2026 - back to the roots](#) - ZLUDA allows to run unmodified CUDA applications on non-NVIDIA GPUs. We now have two new major workl...
3. [OctaneRender: Faqs](#) - OctaneRender requires a CUDA-enabled NVIDIA video card to render. Work is ongoing to assimilate GPUs...
4. [Octane and AMD?](#) - OTOY created a version of OctaneRender for AMD, but it was necessary the full support from AMD for d...
5. [SCALE by Spectral Compute - SCALE Documentation](#) - SCALE is a CUDA-compatible GPU programming toolkit targeting both NVIDIA and AMD GPUs. with zero cod...
6. [Intro to AMD GPU Programming with HIP](#) - HIP provides porting tools which can be used to help port CUDA codes to the HIP layer, with no overh...
7. [Porting CUDA code to HIP - ROCm Documentation](#) - HIP provides a runtime API for AMD GPUs that closely mirrors CUDA, making it straightforward to port...
8. [ROCm vs CUDA: GPU Computing Comparison \(July 2026\)](#) - Compare AMD's ROCm vs NVIDIA's CUDA: performance, costs, and compatibility. CUDA is the industry sta...
9. [Awkward... Maxon releases ProRender, essentially Octane](#) - Octane was built on CUDA so the fact that Otoy will make an AMD compatible version whenever they do ...
10. [Running a Local LLM on AMD Radeon 780M — gfx1103 ...](#) - The Radeon 780M uses GPU architecture version gfx1103. AMD maintains near-binary compatibility across...
11. [ollama-rocm-pangolin15-runbook.pdf](#)
12. [How CUDA Can Run on AMD & Intel GPUs](#) - Discover how ZLUDA enables CUDA apps to run on AMD & Intel GPUs—no rewrites required. Learn the orig...
13. [Open-source project ZLUDA lets CUDA apps run on AMD GPUs](#) - Version 3 of ZLUDA is intended to enable GPU-based applications developed using NVIDIA's CUDA API to...
14. [CUDA emulator for AMD GPUs Zluda loses funding with v6 ...](#) - Zluda loses funding with v6 release — embattled project goes back to hobby status but now includes 3...
15. [NVIDIA CUDA Can Now Directly Run On AMD GPUs Using ...](#) - British startup Spectral Compute has unveiled “SCALE,” a GPGPU toolchain that allows NVIDIA’s CUDA t...

16. [New SCALE tool enables CUDA applications to run on ...](#) - Spectral Compute has introduced SCALE, a new toolchain that allows CUDA programs to run directly on ...
17. [AMD 'Scales' up its CUDA capabilities](#) - Spectral Compute, a London-based software developer, has created Scale, a GPGPU toolchain that allow...
18. [Running GPU-Optimised Monte Carlo \(GOMC\) on an AMD ...](#) - SCALE can be installed by following the official instructions: Install SCALE 1.3.1. I am using neith...
19. [HIP Porting Guide - ROCm Documentation - AMD](#) - HIP is designed to ease the porting of existing CUDA code into the HIP environment. This section des...
20. [Application portability with HIP](#) - This blog discusses various ROCm tools developers can leverage to port existing applications from CU...
21. [Octane and its GPU support: NVIDIA vs. AMD](#) - OctaneRender on Windows and Linux requires a CUDA-enabled NVIDIA GPU to render, but not an OpenCL GP...
22. [NEW: Octane X - is finally here! \[MEGA FAQ THREAD\]](#) - Octane X needs to run on macos 10.15.6. Octane X is a version ahead of CUDA, after OctaneRender 2020...
23. [Blender CUDA on AMD GPUs \[ZLUDA by Vosen\] - Latest News](#) - Rendering results from youtube video: ZLUDA V3.0 (Stable version - official release) : 4 minutes. HI...
24. [Run Ollama on AMD GPU ROCm with TuxedoOS - flyingpenguin](#) - **Step 1: Present “clean” credential**
...
You just bind-mount a temporary
...
**Step 2: Install AMD...
25. [AMD GPUs](#) - Compatible iGPUs include the Radeon 780M (gfx1103) on Phoenix and Hawk Point 7X40 and 8X40 APUs and ...